N

NavigSA

Multilingual Student Support Platform

How the system works: architecture, authentication, data model, security, and deployment.

React 18 | TypeScript | Vite | Supabase | PostgreSQL | Vercel

multilingual-student-platform-navig.vercel.app

THE PROBLEM

Applying from abroad means juggling a dozen systems

- Every university has its own portal, deadlines and document requirements.
- Documents need certified translation before they are accepted.
- Students coordinate all of it by email, in a second or third language.
- There is no single place to see what is done and what is outstanding.

What NavigSA replaces

Spreadsheets tracking deadlines
Email threads with translators and agents
Folders of scanned documents on a laptop
Guessing which requirement comes next

The result

One account. Four languages.
Applications, documents and verified providers
in a single tracked workflow.

WHAT IT DOES

Seven things the platform handles

Accounts

Signup with email confirmation, login, password reset, persistent sessions

Onboarding

Three steps: personal details, ID information, study preferences

Dashboard

Live counts, progress averages, recent documents, verification status

Applications

Create, track, filter and delete, with deadlines and requirement counts

Documents

Drag-and-drop upload to private storage, signed-URL viewing, delete

Marketplace

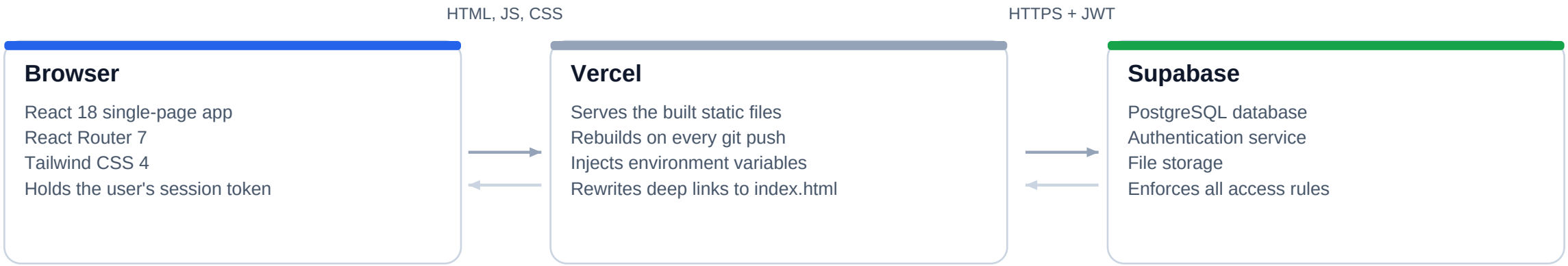
Verified providers by category: translation, visa, housing, tutoring, legal

Multilingual

English, Portuguese, French, Spanish - saved to your profile

ARCHITECTURE

Three tiers, no server of our own



There is no backend code to maintain

The browser talks to Supabase directly. Security is not enforced by an API layer we wrote - it is enforced by the database itself, on every single query.

STARTING POINT

What we actually inherited

A Figma Make export

Every page looked complete.

But all data was hardcoded arrays inside the components, and the login form called navigate() without ever reading the email or password fields.

Nothing was stored. Nothing persisted. Refreshing lost everything.

```
// LoginPage.tsx - as shipped
const handleSubmit = (e) => {
  e.preventDefault();
  if (isLogin) {
    navigate('/dashboard');   // no auth
  } else {
    navigate('/onboarding');
  }
};
```

It was a clickable prototype

A convincing one - but a prototype, not an application.

BEFORE ANY OF THAT

Two defects stopped the project installing at all

1. A corrupt lockfile

71 entries in package-lock.json had no version field at all - the native binaries for esbuild, rollup, lightningcss and tailwind oxide.

npm called semver.eq('') on them and threw:

npm error Invalid Version:

Fix: delete the lockfile, let npm rebuild it.

2. React was never installed

react and react-dom were declared only as optional peerDependencies, so npm skipped them - while 57 files imported React.

Even with a working lockfile the dev server would have failed to resolve 'react'.

Fix: move both into dependencies.

Both were defects in the export itself

Neither was caused by anything the user did. This is a known failure mode of Figma Make exports, and the error messages point nowhere near the real cause.

AUTHENTICATION

What happens when someone signs up

1 Sign up

Email and password sent to Supabase Auth

2 Confirm

Supabase emails a link. No session until clicked

3 Trigger

A database trigger creates the matching profiles row

4 Sign in

Supabase returns a JWT carrying the user's id

5 Session

Token stored, auto-refreshed, survives page reloads

Created by the database, not the app

A Postgres trigger on auth.users fires the moment an account is created, so a profile always exists for every user. The browser cannot forget to create one, and it cannot be skipped.

```
create trigger on_auth_user_created
  after insert on auth.users
  for each row
  execute function handle_new_user();
```

STAYING SIGNED IN

AuthContext holds the session for the whole app

```
// AuthContext.tsx

supabase.auth.getSession()        // on mount: is there a saved session?
  .then(({ data }) => setSession(data.session));

supabase.auth.onAuthStateChange(  // then: react to every change
  (_event, next) => setSession(next)
);
```

Why a context and not props

Every page needs to know who is signed in.
Passing that down by hand through seven pages
would be brittle.

AuthContext exposes one hook:

const { user, profile, signOut } = useAuth()

Session, profile row and auth actions - anywhere.

Three things this gets right

Sessions persist across reloads, so a refresh does not log you out. Tokens refresh automatically before they expire. And a 'loading' flag means the app waits for the session check to finish before deciding whether to redirect - otherwise a signed-in user would be bounced to the login page for a frame.

PROTECTING PAGES

One component guards every private route

```
// ProtectedRoute.tsx

const { session, loading } = useAuth();

if (loading) return <Spinner />;        // still checking
if (!session) return <Navigate to='/' replace />;

return <Outlet />;                      // signed in: render the page
```

Routes are nested inside it

/ login public

/onboarding guarded
/dashboard guarded
/marketplace guarded
/applications guarded
/documents guarded
/profile guarded

This is convenience, not security

A route guard only hides the user interface. Anyone can open the browser console and call the database directly. The real protection is Row Level Security, enforced inside PostgreSQL - which is the next slide, and the most important one in this deck.

DATA MODEL

Four tables

profiles

id -> auth.users
first_name, last_name
phone, country
preferred_language
field_of_study
verification_status
onboarding_completed
notification_prefs

applications

id
user_id
university, program
status, progress
deadline
requirements_total
requirements_completed
next_step

1:N

documents

id
user_id
application_id
name, type
storage_path
size_bytes, mime_type
status
rejection_reason

services

id
name, provider
category
rating, reviews
price, location
verified, featured

Ownership is the organising idea

profiles, applications and documents all carry a user_id pointing at the account that owns them. Every security rule in the system is built on that single column. services is the exception: a shared catalogue nobody personally owns.

Storage

Files live in a private bucket, not in the database. documents stores the path only.

ROW LEVEL SECURITY

The database refuses to return other people's rows

Without RLS

The browser holds a key that can read the whole table. Security depends on every query in the app remembering to filter correctly.

One forgotten .eq('user_id', me) and every student's documents are exposed.

The app is the only thing standing in the way.

With RLS

Postgres attaches the rule to the table itself.

Every query - from our code, from the console, from curl - is silently rewritten to include 'and user_id = the caller's id'.

Forgetting to filter is no longer possible. The wrong rows simply do not come back.

Demonstrated in production

Signed out, a request for the services table returns an empty array - not an error, just nothing. Signed in, the same request returns all five providers. Identical code, identical key; the database decides based on who is asking.

ROW LEVEL SECURITY

What the policies actually look like

```
-- turn it on: the table now denies everything by default
alter table public.documents enable row level security;

-- then grant back, narrowly
create policy "read own documents"
  on public.documents for select
  using (auth.uid() = user_id);

create policy "insert own documents"
  on public.documents for insert
  with check (auth.uid() = user_id);
```

auth.uid()

Postgres reads the user id straight out of the signed JWT on the request.

It cannot be spoofed from the browser - the token is signed by Supabase and verified server-side.

using = which rows you may read
with check = which rows you may write

Deliberately not self-serve

documents.status and profiles.verification_status should be set by a reviewer, not by the student who uploaded the file. Those writes belong to the service_role key on a server - which bypasses RLS entirely and must never appear in browser code.

DOCUMENT STORAGE

Private files, scoped by user id

1. Upload

Browser sends the file to the documents bucket at the path:

<user-id>/<filename>

2. Policy check

Storage compares the first folder in the path against auth.uid().

Mismatch: rejected.

3. Record

A documents row stores the path, size, type and status.

Upload fails: row rolled back, file removed.

4. Viewing

No public URL exists.

A signed URL is minted on demand and expires after 60 seconds.

Why the path matters

The bucket is private, so there is no guessable public link. The storage policy pins the first path segment to the caller's user id, which means one student cannot reach another student's file even knowing its exact name. The bucket also rejects anything over 10 MB or outside PDF, JPEG and PNG - enforced server-side, so bypassing the browser check achieves nothing.

ANATOMY OF A PAGE LOAD

Opening the Documents page

1	**Route guard**	ProtectedRoute confirms a session exists, or redirects to login.
2	**Hook fires**	useAsyncData calls listDocuments(user.id) and sets loading = true.
3	**Query sent**	api.ts asks Supabase for documents joined to their application.
4	**RLS applies**	Postgres silently scopes the query to rows owned by this user.
5	**Render**	Rows arrive, loading flips to false, the list and counters draw.
6	**On demand**	Clicking View mints a 60-second signed URL for that one file.

Stale responses are discarded

useAsyncData tracks whether a request is still current, so a slow first query cannot overwrite the result of a faster second one.

HOW THE CODE IS ORGANISED

Four layers, each with one job

Pages

`Login, Onboarding, Dashboard, Marketplace, Applications, Documents, Profile`

Rendering and local UI state only

Contexts

`AuthContext, LanguageContext`

Shared state every page needs: session, profile, translations

Data layer

`api.ts, useAsyncData.ts`

Every database and storage call lives here - pages never query directly

Client

`supabase.ts, database.types.ts`

One configured client, one set of table types

Why the data layer is separate

Changing a table means editing api.ts, not hunting through seven page components. It is also the only place a query can be got wrong, so it is the only place to audit.

MULTILINGUAL SUPPORT

Four languages, switchable anywhere

```
// LanguageContext.tsx

const translations = {
  en: { 'login.title': 'Welcome Back',  ... },
  pt: { 'login.title': 'Bem-vindo de volta', ... },
  fr: { 'login.title': 'Bon retour',    ... },
  es: { 'login.title': 'Bienvenido',    ... },
};

const t = (key) => translations[language][key] || key;
```

How it behaves

Components call t('login.title') rather than holding English strings.

The chosen language is written to the user's profile, so it follows them to any device.

On sign-in the saved language is restored - but a choice made in the header this session is never overwritten by the loading profile.

The honest limitation

Translations are a hardcoded dictionary in one file, which is fine at this size but does not scale to hundreds of strings or to translators who are not developers. A production system would move these into per-language JSON files, or a library such as i18next.

DEPLOYMENT

Pushing to GitHub publishes the site

git push

Commit lands on the branch

Vercel builds

npm install, npm run build

Env injected

Supabase URL and anon key compiled in

Deployed

Static files on a global CDN

Live

Reachable at the production URL

Deep links need a rewrite

React Router uses real URLs, so refreshing on /dashboard asks the host for a file that does not exist. vercel.json rewrites every non-asset path to index.html - while still letting a genuinely missing asset return a real 404.

Secrets never enter the repository

.env.local is gitignored. The keys live in Vercel's environment settings and are injected at build time. Only the anon key is ever used in the browser - it is safe there precisely because RLS constrains it.

ONE SUBTLETY WORTH KNOWING

Environment variables are compiled in, not read at runtime

What people expect

That the running site reads its configuration when a visitor loads the page - so changing a variable takes effect immediately.

What actually happens

Vite replaces import.meta.env.VITE_SUPABASE_URL with the literal string during the build.

The value is baked into the JavaScript bundle. Changing it later does nothing until you rebuild and redeploy.

Two consequences

First: environment variables must be set before the first deploy, or the live site ships with undefined credentials and shows the 'not connected' banner. Second: anything in a VITE_ variable is readable by anyone who opens the bundle - which is exactly why the service_role key must never be one, and why the anon key is safe only because Row Level Security stands behind it.

HOW WE KNOW IT WORKS

Checks run against the live system

Type safety	tsc --noEmit passes across 89 files under strict mode
Build	Production build succeeds: 1,667 modules, 154 KB gzipped
Deep links	/dashboard, /documents and /profile all return 200, not 404
Asset 404s	A missing bundle still returns 404 - the rewrite does not mask it
Auth endpoint	Supabase auth and REST endpoints both return 200
RLS, signed out	Query for services returns zero rows
RLS, signed in	Same query returns all five providers on the live dashboard
Persistence	Onboarding data written to profiles and read back after sign-in

LIMITATIONS

What is deliberately not finished

Still user-interface only

No reviewer workflow - documents are marked verified from the Supabase dashboard

Marketplace Contact and favourite buttons

WhatsApp Connect Now on the dashboard

Two-factor authentication enrolment

Would need attention at scale

Translations hardcoded in one file

554 KB bundle - worth code-splitting

Students can currently update their own documents row, which includes status; a column-level grant closes that

Supabase's shared email sender is rate-limited

Naming these is the point

Every one of these is a known, bounded gap with a clear fix - not an unknown. The schema already supports the reviewer workflow; what is missing is the interface on top of it.

IN SUMMARY

From clickable prototype to working system

- A Figma export with hardcoded arrays became an application with a real database behind it.
- Authentication, sessions and route guarding are handled end to end.
- Security is enforced by PostgreSQL, not by the front-end code.
- Files are private, scoped per user, and served through expiring links.
- Every push to GitHub redeploys the live site automatically.

The idea worth keeping

The browser holds a key that anyone can read.

That is safe only because the database refuses to answer questions the caller is not entitled to ask.

Security belongs in the data layer, not in the interface that happens to sit in front of it.

Thank you

Questions?

LIVE SITE

multilingual-student-platform-navig.vercel.app

SOURCE

github.com/MarcCput/Multilingual-Student-Platform-NavigSA-